

8-Puzzle Problem using Iterative Deepening Search (IDS)

Start: 2 8 3 / 1 6 4 / 7 _ 5 Goal: 1 2 3 / 8 _ 4 / 7 6 5

1. Python Code

```
"""8-Puzzle solver using Iterative Deepening Search (IDS).

IDS repeatedly runs Depth-Limited DFS with limit = 0, 1, 2, ...
until the goal is found. It is complete and finds the shallowest
(optimal in number of moves) solution, using only O(d) memory.
"""

START = (2, 8, 3,
         1, 6, 4,
         7, 0, 5)

GOAL = (1, 2, 3,
        8, 0, 4,
        7, 6, 5)

MOVES = {"UP": -3, "DOWN": 3, "LEFT": -1, "RIGHT": 1}

def neighbors(state):
    """Yield (action, new_state, swapped_tile) for every legal blank move."""
    b = state.index(0)
    row, col = divmod(b, 3)
    for action, delta in MOVES.items():
        if action == "UP" and row == 0:
            continue
        if action == "DOWN" and row == 2:
            continue
        if action == "LEFT" and col == 0:
            continue
        if action == "RIGHT" and col == 2:
            continue
        n = b + delta
        new = list(state)
        new[b], new[n] = new[n], new[b]
        yield action, tuple(new), state[n]

def depth_limited_search(state, goal, limit, path, stats):
    """DFS up to 'limit'. 'path' holds states on the current branch (cycle check)."""
    stats["expanded"] += 1
    if state == goal:
        return []
    if limit == 0:
        return None
    for action, nxt, tile in neighbors(state):
        if nxt in path:
            # avoid cycles on the current path
            continue
        path.add(nxt)
        result = depth_limited_search(nxt, goal, limit - 1, path, stats)
        path.remove(nxt)
        if result is not None:
            return [(action, tile, nxt)] + result
    return None

def ids(start, goal, max_depth=30):
    """Return (solution_steps, per_iteration_stats)."""
    log = []
    for limit in range(max_depth + 1):
        stats = {"expanded": 0}
        result = depth_limited_search(start, goal, limit, {start}, stats)
        log.append((limit, stats["expanded"], result is not None))
        if result is not None:
            return result, log
    return None, log

def board_str(state):
    sep = "  +---+---+---+"
    lines = [sep]
    for r in range(3):
        cells = ["  " if v == 0 else f" {v}" for v in state[r * 3:r * 3 + 3]]
```

```

        lines.append("  |" + "|".join(cells) + "|")
        lines.append(sep)
    return "\n".join(lines)

def print_step(step, state, depth, action=None, tile=None, goal=False):
    bar = "=" * 43
    if step == 0:
        title = "INITIAL STATE SETUP"
    elif goal:
        title = f"MOVE BLANK {action} (GOAL STATE REACHED!)"
    else:
        title = f"MOVE BLANK {action} (SWAPPED WITH {tile})"
    print(bar)
    print(f"  STEP {step} : {title}")
    print(bar)
    print(board_str(state))
    print()
    print(f"  Depth in search tree = {depth}")
    print()
    print("  " + "-" * 43)
    print()

if __name__ == "__main__":
    solution, log = ids(START, GOAL)

    print("ITERATIVE DEEPENING SEARCH - SEARCH LOG")
    print("=" * 43)
    print("  Depth Limit | Nodes Expanded | Goal Found")
    print("  " + "-" * 41)
    for limit, nodes, found in log:
        print(f"    {limit:^11} | {nodes:^14} | {'YES' if found else 'no'}")
    print()

    print_step(0, START, 0)
    for i, (action, tile, state) in enumerate(solution, start=1):
        print_step(i, state, i, action, tile, goal=(state == GOAL))

    print("#" * 55)
    print(f"  FINAL ANSWER: Goal reached successfully at Step {len(solution)}!")
    print(f"  Solution depth = {len(solution)}")
    print(f"  Total nodes expanded over all iterations = {sum(n for _, n, _ in log)}")
    print(f"  Moves: " + " -> ".join(a for a, _, _ in solution))
    print("#" * 55)

```

2. Output

```
print("#" * 55)
```

... ITERATIVE DEEEPENING SEARCH - SEARCH LOG

=====

Depth Limit	Nodes Expanded	Goal Found
0	1	no
1	4	no
2	9	no
3	19	no
4	33	no
5	7	YES

=====

STEP 0 : INITIAL STATE SETUP

=====

+---+---+---+
2 8 3
+---+---+---+
1 6 4
+---+---+---+
7 5
+---+---+---+

Depth in search tree = 0

=====

STEP 3 : MOVE BLANK LEFT (SWAPPED WITH 2)

=====

+---+---+---+
2 3
+---+---+---+
1 8 4
+---+---+---+
7 6 5
+---+---+---+

Depth in search tree = 3

=====

STEP 4 : MOVE BLANK DOWN (SWAPPED WITH 1)

=====

+---+---+---+
1 2 3
+---+---+---+
8 4
+---+---+---+
7 6 5
+---+---+---+

Depth in search tree = 4

=====

Hello, Dehini

How can I help you today?

How can I install Python libraries?

Load data from Google Drive

Show an example of training a simple ML model

What can I help you build?

=====

STEP 5 : MOVE BLANK RIGHT (GOAL STATE REACHED!)

=====

```
+---+---+---+
| 1 | 2 | 3 |
+---+---+---+
| 8 |   | 4 |
+---+---+---+
| 7 | 6 | 5 |
+---+---+---+
```

Depth in search tree = 5

#####

FINAL ANSWER: Goal reached successfully at Step 5!

Solution depth = 5

Total nodes expanded over all iterations = 73

Moves: UP -> UP -> LEFT -> DOWN -> RIGHT

#####
